nTop-AVL Integration

Tianxiang Gu

June 11, 2026

Introduction

This document aims to provide insight into integrating the vortex lattice method solver, AVL, into nTop. A general framework is provided, where the parameters governing the aircraft's geometry can be freely changed by the user. This feature enables automatic computation of aerodynamic coefficients, stability derivatives and static margin when aircraft parameters or flight conditions are updated.

The example aircraft used here is a simple high-wing, taildragger aircraft. It is shown below in Figure 1:

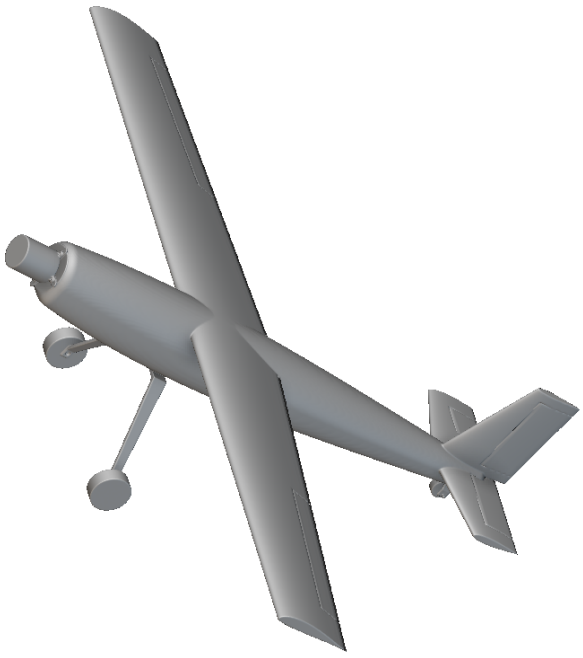

Figure 1: Geometry of the example aircraft rendered in nTop.

nTop Setup

The main requirements within nTop is that it must contain the necessary parameters for the Python script to successfully execute. In this example, we define the aircraft geometry using the following parameters, also using them as inputs to the Python script:

Parameter	**Value**
Wing Area	0.136m^2
Wing Aspect Ratio	8.11
Wing Taper Ratio	0.75
Quarter Chord Sweep Angle	$-1.01°$
Wing Vertical Position	0.025m
Horizontal Stabiliser Area	0.021m^2
Horizontal Stabiliser Aspect Ratio	3
Horizontal Stabiliser Taper Ratio	0.7
Horizontal Stabliser Incidence Angle	$-3°$
Horizontal Stabilier Vertical Position	0.01m
Vertical Stabiliser Area	0.0125m^2
Vertical Stabiliser Aspect Ratio	1.5
Vertical Stabiliser Taper Ratio	0.6
Vertical Stabiliser Vertical Position	0m
Tail Moment Arm	0.4m
CG Position (3 separate inputs)	(0.039m, 0m, 0m)
Aileron Start/End Span %	0.5, 0.9
Aileron Chord %	0.25
Elevator Start/End Span %	0, 0.8
Elevator MAC %	0.25
Rudder Start/End Span %	0.1, 0.9
Rudder Chord %	0.3
Mach Number	0.052
Alpha	$2°$

Note that if control surfaces are not of interest, chord/span fractions can be left out of the parameterisation. These parameters are not necessarily inputs either.

Python Script

The Python script is crucial in taking inputs from nTop parameters and creating a geometry file that is ready to use by AVL. In this example, the Python file retrieves aircraft parameters, creates a `.avl` geometry and executes an AVL run. The code consists of three main sections: initial definition, run execution and results parsing. The driver script takes inputs and computes necessary derived geometry values, such as tail root position and wingspan.

Within the initial definition phase, the input parameters are used to generate a geometry file by defining discrete cross-sections of the various lifting surfaces. To satisfy physical manufacturing constraints, the geometry defines a wing with a straight leading edge using a negative quarter-chord sweep angle, and elevators with a straight, non-binding hinge line. This straight hinge is achieved by dynamically calculating a uniform physical elevator width c_e and adjusting the fractional hinge location across the horizontal stabilizer span according to the local chord c_{local}:

$$X_{\text{hinge}} = 1.0 - \frac{c_e}{c_{\text{local}}}$$

A text file containing run commands is also created to supply AVL with the correct keyboard inputs for setting run conditions, including but not limited to angle of attack (α), velocity, and control surface deflections. The text file also handles saving the stability files used to evaluate static stability (dynamic stability analysis is omitted from this particular workflow).

Next, the geometry and commands are fed into AVL and the case is executed. In this example, the AVL instructions include trimming the elevators until steady, level flight is maintained at the given flight conditions. In addition, target roll/pitch/yaw rates were also fed in, and AVL is able to compute the necessary deflection angles of each control surface for achieving the target rates.

Outputs stored in `.st` files are eventually parsed and printed to the command line once the run completes. We included $C_L, C_D, C_{m\alpha}, C_{mq}, C_{n\beta}, C_{l\beta}, C_{lp}$, static margin based on the input CG and control surface deflection angles as outputs here. However, this is highly customisable: effectively anything that AVL is able to save as a results file can be parsed in the script.

Using nTop's run command feature, this string, stored as a variable, can be run. A directory should be specified if the nTop notebook is not placed in the same location as the AVL Python script. The following section will delve deeper into the creation of this string.

Integration

In order to run the Python script within nTop, we must compile a string consisting of all necessary parameters. This is done using the concatenate text block. Most, if not all parameters should be variables defined in nTop to ensure that the run command actively adapts to changes in geometry. For the example case, the string to execute the command is:

```
python AVL_prototype.py --wing_area 0.136 --wing_ar 8.11
--wing_taper 0.75 --wing_sweep -1.01 --z_wing 0.025
--hstab_area 0.021 --hstab_ar 3 --hstab_taper 0.7
--z_htail 0.01 --hstab_incidence -3 --vstab_area 0.0125
--vstab_ar 1.5 --vstab_taper 0.6 --z_vtail 0
--tail_moment_arm 0.4 --x_cg 0.0391 --y_cg 0 --z_cg 0
--aileron_y_start_frac 0.5 --aileron_y_end_frac 0.9
--aileron_chord_frac 0.25 --elevator_y_start_frac 0
--elevator_y_end_frac 0.8 --elevator_chord_frac 0.3
--rudder_z_start_frac 0.1 --rudder_z_end_frac 0.9
--rudder_chord_frac 0.3 --mach 0.06122
--alpha 3 --roll_rate 180 --pitch_rate 180 --yaw_rate 360
```

The geometry created by the run is shown below in Figure 2:

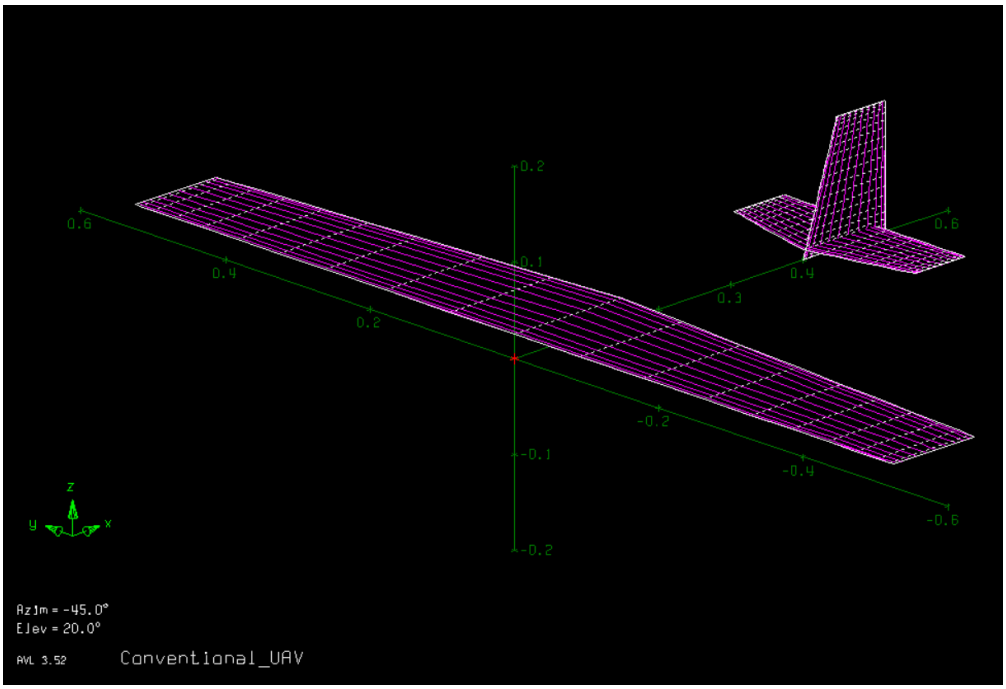

Figure 2: AVL geometry generated by the Python code based on the input line.

Upon running the script, the following output is produced:

```
0.56028,0.01325,−0.0,−0.945608,−10.131258,0.072859,0.002757,
−0.505093,18.79,−0.79755,7.88752,−6.95597,12.81725
```

Parameter	**Value**
C_L	0.56028
C_D	0.01325
C_m	0 (trimmed)
$C_{m\alpha}$	-0.94561
C_{mq}	-10.1313
$C_{n\beta}$	0.07286
$C_{l\beta}$	0.00276
C_{lp}	-0.50509
SM (%)	18.79
Aileron Deflection (deg)	7.888
Elevator Deflection (deg)	-6.956
Rudder Deflection (deg)	12.817

This string of text can be extracted in nTop as part of the properties in the run command block. We can then separate the text with the split text block, using comma as a delimiter and save each numerical value as their corresponding variable.

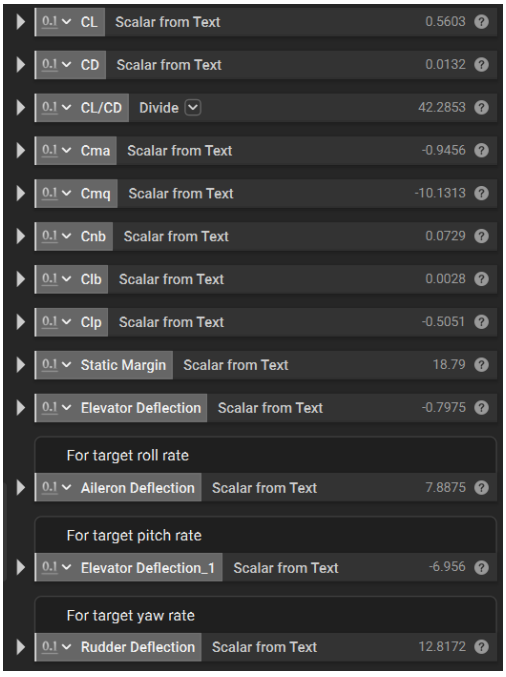

Figure 3: Sample AVL to nTop output